

LLM-Powered Knowledge Graph Extractor with Provenance, Event Reification, and Importance-Ranked Visualization

Mohamad Jehad Motaz Khachfa
800241913

CS585 Knowledge Engineering
mohamadjehad@aucegypt.edu

Abstract

Building useful knowledge bases from text has been a challenge for a long time. The standard process, known as the Knowledge Acquisition Bottleneck, requires a domain expert to read documents and write down entities and relations by hand. Large Language Models (LLMs) can now do most of this work in a single API call. This paper describes a web application that takes any text and returns an interactive knowledge graph. The system has four main design choices. First, it uses an open vocabulary for entity types with a controlled vocabulary of 23 relations. We started with 15 relations (the typical encyclopedic set) and added 8 more after we noticed that many biographical facts were collapsing to a generic `related_to` edge. Second, the prompt tells the model to use *event reification* for biographical events such as births and marriages, so that a person is never directly connected to a date. Third, the prompt asks the model to merge surface mentions of the same entity using an `aliases` field. Fourth, every edge carries a verbatim evidence sentence, a self-reported confidence in $[0, 1]$, and an explicit `[FALLBACK]` marker when the generic `related_to` predicate was used. The visualization shows confidence as a dashed line and fallbacks as a dotted italic line, so the user can see directly where the vocabulary fails. The frontend also includes an *importance-ranked complexity slider* that hides less central entities at lower levels, which is useful for paper-ready summary views. The system works with four LLM providers (Anthropic Claude, OpenAI GPT, Google Gemini, and local Ollama models) behind one shared schema, and exports the graph as JSON, PNG, or RDF/Turtle.

1 Introduction

Building queryable knowledge bases from text has historically been slow and expensive. human experts need to read every document, decide which entities and relations are relevant, and write them down. Feigenbaum [1] called this the *Knowledge Acquisition Bottleneck*, and the problem has not really disappeared. Tools like Protégé and CycL help with modeling, but they do not remove the manual reading step.

LLMs change this in a special way. Because they are trained on a very large amount of text, they can already recognize entities and relations in any new passage without being trained on a specific datasets. They can also be asked to return structured output (JSON with a fixed schema) using tool-use APIs [11] or strict JSON-Schema decoding [12]. The natural question is whether this can be turned into a usable application that does most of what a knowledge engineer used to do by hand.

This paper describes such an application. The first version (v1) was a thin wrapper around an LLM call: one prompt, 15 relations, a force-directed graph view. While testing it on biographical and scientific passages we found several problems that match well-known issues from the knowledge engineering literature: missing relations, generic fallbacks, type errors (a person “located in”

a year), and duplicated entities for the same real-world thing. The second version (v2), described here, fixes these problems with a longer system prompt, an extended relation list, an `aliases` field, an `[FALLBACK]` marker, and several new visualization controls.

The main contributions of this work are:

1. A hybrid extraction schema (Section 4) with open entity types and 23 controlled relations, chosen by looking at where v1 was falling back to `related_to`.
2. A decision-rule system prompt (Section 5) that tells the model which relation to use for each common pattern, and that asks it to do event reification, alias merging, and generic/specific deduplication during extraction.
3. A per-edge audit trail (Sections 4, 6): every edge has a verbatim evidence sentence, a confidence value in $[0, 1]$, and a `[FALLBACK]` prefix when the model used the generic relation.
4. An interactive visualization (Section 6) with degree-based node sizing, an ego-network highlight on hover, an importance-ranked complexity slider, and several filters that compose together.
5. A provider-agnostic backend (Section 3) covering Anthropic, OpenAI, Gemini, and Ollama behind the same schema, with RDF/Turtle export.

The rest of the paper is structured as follows. Section 2 reviews related work. Section 3 describes the architecture. Section 4 explains the knowledge model and Section 5 the prompt. Section 6 describes the visualization. Section 7 reports a qualitative evaluation. Section 8 discusses limitations. Section 9 concludes.

2 Related Work

Automated knowledge graph construction has gone through three main phases, and a few other research areas are also relevant for the choices we make below.

Rule-based and supervised pipelines. Early systems combined Named Entity Recognition with a separately trained Relation Extraction model. Mitchell et al.’s NELL system [2] showed that a semi-automated pipeline can keep building a knowledge base for years, but it still needed regular human curation, and it did not generalize well across domains. Adding a new relation meant collecting and labeling new training data.

Knowledge graph embeddings. A parallel line of work focused on learning vector representations for entities and relations. TransE [3] is the classic example: $\text{head} + \text{relation} \approx \text{tail}$ in the embedding space. Later models such as RotatE and ComplEx improved the geometry. These methods are mainly for reasoning over an existing graph (predicting missing links, finding inconsistencies). They are not directly useful for building a graph from raw text.

LLM-based extraction. Recent work uses LLMs as zero-shot or few-shot extractors. Pan et al. [4] survey this area and argue that LLMs and KGs should be combined: LLMs provide natural-language understanding, KGs provide structure and queryability. They specifically recommend controlled relations with open entity types, which is the choice we follow in Section 4. Microsoft’s GraphRAG [5] combines LLM extraction with graph-aware retrieval to improve question answering, and industry tools such as Neo4j’s LLM Graph Builder [6] package the same idea for users. Our system fits in this third phase, but we focus on three things that other tools handle less explicitly: vocabulary coverage as an auditable property, n-ary reification done inside the prompt, and an importance-based summary view.

Ontology design and n-ary relations. Noy and McGuinness’s *Ontology Development 101* [7] explains the standard way to handle relations that have more than two arguments: introduce an extra node for the event itself, and attach the arguments to that node. A marriage, for example, involves two people, a date, and sometimes a place. In our system, the prompt asks the LLM to do this reification at extraction time rather than as a post-processing step.

Entity resolution. Ji et al. [8] survey knowledge graph construction and completion. They list entity resolution (deciding when two surface forms refer to the same entity) as one of the recurring failure modes of automated pipelines. The Maria Skłodowska / Marie Curie case from their survey is exactly the problem we fix in v2 with the `aliases` field (Section 5.3).

Knowledge graph summarization. Visualization research has noticed for a long time that even small KGs become hard to read after a few dozen nodes. Liu et al. [9] review KG construction techniques and discuss summarization options based on structural features (degree, centrality) or semantic ones (query relevance, ontology depth). Our complexity slider (Section 6.4) is structural: it ranks entities by degree plus mention frequency, which is simple to compute and easy to explain.

Provenance. W3C’s PROV-O [10] and related work all stress that being able to trace a fact back to its source is necessary for trustworthy reuse. We follow a lightweight version of this: every node carries source sentences, every edge carries an evidence sentence and a confidence score, and the RDF export keeps these using standard `rdf:Statement` reification.

3 System Architecture

The project is a small monorepo with two parts: a stateless Express backend that exposes the extraction API, and a Next.js frontend that handles the input, the visualization, and the export. They talk over JSON HTTP; the frontend proxies `/api/*` to the backend during development. Figure 1 shows the high-level flow.



Figure 1: High-level data flow. The frontend never talks to LLM providers directly. The backend chooses the right adapter and normalizes the result into a single `{nodes, edges, meta}` schema. Only one LLM call is made per extraction; everything else (filtering, layout, complexity slider, export) happens in the browser.

3.1 Provider Adapters

Each LLM is wrapped in an adapter with the same interface: `isConfigured()` and `extract(text, model)`. Each adapter uses the most reliable structured-output mechanism that provider supports:

- **Anthropic Claude:** one forced tool call, `tool_choice = {type: "tool", name: "emit_graph"}`, with the JSON Schema as the tool’s `input_schema`. The adapter unwraps the `tool_use` block.

- **OpenAI GPT:** structured outputs with `response_format = {type:"json_schema", strict:true}`. The API enforces the schema, but it requires every property to be in required. That is why we make aliases and confidence mandatory rather than optional.
- **Google Gemini:** `responseSchema` in `generationConfig` with `responseMimeType:"application/json"`. Gemini uses its own `SchemaType` enum, so we keep a parallel schema definition that mirrors the canonical JSON Schema.
- **Ollama:** the local-model adapter uses `format:"json"` on the `/api/chat` endpoint. Open models do not yet enforce schemas as strictly as commercial APIs, so the post-extraction normalizer does more work here.

A central registry auto-detects which adapters are configured (by checking environment variables for the commercial APIs, and by pinging `localhost:11434` for Ollama) and shows only those in the UI. This way the app stays usable even when the user has only one provider.

3.2 Endpoints

The backend exposes three endpoints:

- GET `/api/providers`: returns the list of usable providers and their model menus.
- POST `/api/extract`: takes `{text, provider, model}` and returns `{nodes, edges, meta}`.
- POST `/api/export/turtle`: takes a graph object and returns an RDF/Turtle file. The serialization uses `rdf:Statement` reification so that evidence and confidence survive the export.

Between the adapter and the response there is a normalizer (`normalizeGraph` in `schema.js`). It drops nodes that do not have an `id` or a `label`, drops edges that point to a node that does not exist, replaces unknown relations with `related_to`, and clamps any confidence value into `[0, 1]` with a default of 0.7 when the field is missing.

4 Knowledge Model and Schema Design

The most important design choices in this kind of system are the knowledge model (what counts as an entity, what counts as a relation, what extra fields each one carries) and the prompt that asks for it. We discuss the model here and the prompt in Section 5.

4.1 Hybrid Schema: Open Types, Controlled Relations

A knowledge model can be *closed*, where both entity types and relation labels are fixed in advance, or *open*, where the extractor invents them. Closed models give clean, queryable graphs but only work in one domain. Open models handle any text but suffer from *label drift*: the same relation appears as *works_at*, *employed_by*, and *is_employee_of* in different runs, which breaks SPARQL queries and any union operation.

We use a *hybrid*: open entity types, but controlled relations. The model picks the most natural type for each entity (Person, Element, Theory, Field of study, etc.), while relation labels come from a fixed list. This split is also what Pan et al. [4] recommend.

4.2 Extended Relation Vocabulary (v2)

The first version of the vocabulary had 15 relations covering the encyclopedic baseline. After running v1 on three biographical and scientific passages, we noticed that certain patterns kept ending up as `related_to`. The three biggest categories were:

- *Biographical place relations.* A person is *born in* a city, or *lived in* a country. The v1 vocabulary only had `located_in`, which makes sense for organizations but is a type error for people.
- *Discovery vs. creation.* Natural elements, particles, and laws are *discovered*, not *created*. v1 only had `created_by` and `developed_by`, which both imply human authorship and so are wrong for elements like polonium.
- *Shared honours.* When a Nobel Prize is shared between three people, the symmetric co-recipient relation was lost as three separate `related_to` edges.

So we extended the vocabulary to 23 relations. The new ones are added specifically to cover the cases above:

Original (v1): `works_for`, `member_of`, `founded_by`, `located_in`, `headquartered_in`, `part_of`, `created_by`, `developed_by`, `causes`, `influences`, `occurred_at`, `occurred_on`, `has_property`, `related_to`, `opposes`.

Added in v2: `discovered_by`, `born_in`, `lived_in`, `married_to`, `studied_at`, `awarded`, `awarded_for`, `co_recipient_with`.

The way we picked the new relations is the same idea as a normal vocabulary-coverage audit in ontology engineering: look at the fallbacks, find the most common missing patterns, add relations for those. The [FALLBACK] marker in Section 5.5 makes this audit easy to do on any new domain.

4.3 Output Schema

The extraction target is a JSON object with two arrays. Listing 1 shows the v2 schema (abbreviated). The v2 additions are `aliases` on nodes and `confidence` on edges.

Listing 1: .]v2 output schema (abbreviated). Each node carries its source sentences and any aliases. Each edge carries the verbatim evidence sentence and a confidence value in [0,1].

```
{
  "nodes": [{
    "id": "marie_curie",
    "label": "Marie Curie",
    "type": "Person",
    "description": "Polish-born French physicist and chemist...",
    "source_sentences": ["Marie Curie was a Polish-born physicist..."],
    "aliases": ["Maria Skłodowska"] // v2
  }],
  "edges": [{
    "source": "polonium",
    "target": "marie_curie",
    "relation": "discovered_by", // v2 relation
    "evidence": "...they discovered the elements polonium and radium.",
    "confidence": 0.95 // v2
  }]
}
```

Three things to note. First, every node has source sentences and every edge has an evidence sentence. This lets the user check the extraction, and it also makes the model less likely to invent facts: when you ask the model to cite a sentence, it tends to stay closer to the text. Second, the `id` field uses `lowercase_with_underscores`, which works directly as a URI fragment in the RDF/Turtle export (`kg:marie_curie`). Third, `aliases` and `confidence` are required (not optional) in the schema, because OpenAI’s `strict:true` mode does not allow optional properties; the model returns `[]` or a default of 0.7 when there is nothing to say.

5 The Prompt

The system prompt is the same for all four providers; only the way it is delivered changes between SDKs. Moving from v1 to v2 we added five things to the prompt, each one motivated by a specific problem we saw in v1.

5.1 Decision Rules for Each Relation

The v1 prompt listed the relations and said “use the one that fits, or fall back to `related_to`”. This led to a lot of falling back, because the model was not given clear examples of when each relation applied. The v2 prompt adds short decision rules for the new relations, and explicitly forbids the wrong alternative. For example:

A PERSON born in a place → born_in (NEVER located_in for people).
A PERSON living or working in a place → lived_in.
An ORGANIZATION sited in a place → located_in or headquartered_in.
A NATURAL phenomenon, element, law, or species identified by a scientist → discovered_by.
Use created_by / developed_by only for things humans MADE.
Two or more people sharing an award → co_recipient_with between each pair (sym-metric; emit both directions).

The result is a much longer prompt (about 1,000 tokens) but the extra cost is small on commercial APIs (under \$0.003 per call on Claude Sonnet), and the extraction quality goes up clearly.

5.2 Event Reification

A biographical event such as a birth or a marriage usually has more than two arguments: a person, a date, sometimes a place, sometimes another person. In an entity–edge graph each edge is binary, so attaching all of them directly to one person produces type errors like `marie_curie occurred_on 1867`, which literally reads as “Marie Curie happened on 1867”.

The standard fix from Noy and McGuinness [7] is to add an intermediate node for the event itself, and attach the arguments to that node:

```
birth_marie_curie occurred_on 1867
birth_marie_curie occurred_at warsaw
birth_marie_curie has_property marie_curie
```

The v2 prompt asks the model to do this reification at extraction time, with short examples for births, marriages, and foundings. The cost of this is real: the graph has more nodes, and some of them are not real entities but “connector” nodes for the events. We deal with this on the visualization side by rendering event nodes smaller and less prominent, and by hiding them first when the complexity slider is moved down (Section 6.4).

5.3 Entity Resolution via Aliases

Encyclopedic text often uses several surface names for one entity. Common cases: “Maria Skłodowska” / “Marie Curie”, “Mark Twain” / “Samuel Clemens”, “Microsoft” / “MSFT” / “the company”. A naive extractor produces two or three disconnected nodes. A correct one merges them.

We add an `aliases: string[]` field to each node and tell the model:

If the text refers to the same entity by multiple names, emit ONE node with the canonical label and list the alternatives under `aliases`. Do NOT create two separate nodes for the same referent.

In the side panel of the visualization, the aliases appear as small italic pills under the main label. The RDF export writes each alias as a `kg:alias` triple, which keeps the canonical form usable in downstream queries.

5.4 Deduplicating Generic and Specific Mentions

A related problem is when both a class and an instance are mentioned: “the Nobel Prize” in one sentence, “the Nobel Prize in Physics” and “the Nobel Prize in Chemistry” in the next. The default behavior is to emit three nodes, with the generic one acting as a phantom hub that adds no real information. The v2 prompt asks the model to prefer specific instances and to link them with `part_of` to the class node only if the class is interesting enough to keep on its own.

5.5 Confidence and the [FALLBACK] Marker

The most useful v2 addition for the user is the audit trail. Each edge has a `confidence` value in $[0, 1]$. The prompt explains the scale: 1.0 for relations stated with an unambiguous verb, 0.7 for relations that are clearly implied but not literally stated, 0.4 for relations that are inferred from broader context. In the visualization, edges with confidence below 0.6 render as dashed lines, so the user can see at a glance which edges are well supported and which are weaker.

In parallel, whenever the model uses the generic `related_to` relation it must start the evidence string with the literal token `[FALLBACK]`. These edges render as dotted, italic, muted lines and carry a `fallback` badge in the side panel. This is the main reason we added the marker: the visualization now *shows* the user where the controlled vocabulary fails, which is the information they need to decide whether to extend it. A graph dominated by dotted edges means the current vocabulary is wrong for this domain.

5.6 RDF/Turtle Export

For interoperability with the Semantic Web stack, the system serializes the graph as Turtle using two custom namespaces (`kg:` for entities, `rel:` for relations), `rdfs:label` and `rdfs:comment` for human-readable strings, `kg:alias` triples for each alias, and `rdf:Statement` reification carrying `kg:evidence` and `kg:confidence` as `xsd:decimal`. This is a deliberately simple mapping: it keeps provenance and quality information, it stays human-readable, and it loads directly into triple stores like GraphDB or Apache Jena.

6 Interactive Visualization

The frontend is a single Next.js page split into two columns. The left column has the text input, the provider and model menus, and three sample paragraphs. The right column has the graph itself, drawn with Cytoscape.js, plus a toolbar above it, a small floating legend at the bottom-left, and a side panel that opens when a node is clicked.

6.1 Visual Encoding

Node color encodes the entity type using a small fixed palette: Person, Organization, Location, Concept, Event, and a default for everything else. Node *size* is proportional to degree, between 28 and 60 pixels, which gives an immediate visual cue for which entities are central. Edge style encodes confidence: solid line for high-confidence vocabulary edges, dashed line for low-confidence

(< 0.6), and dotted-italic-muted for [FALLBACK] edges. Event-reification nodes are rendered smaller, semi-transparent, with a dashed border, so they visually fade behind the real entities.

Edge labels are hidden by default. They only appear for the hovered or selected ego network, or when the user turns on the `labels: all` toggle. This was one of the biggest improvements over v1: with event reification adding more edges, showing every label at once made the graph unreadable.

6.2 Layouts

Five layouts are available behind a single segmented control: force-directed (cose), concentric (rings by degree), hierarchy (breadth-first from the highest-degree node), circle, and grid. When a graph loads, the page picks a default automatically: if one node has at least 40% of the possible degree share (a typical hub, like Marie Curie in a biographical paragraph), the default is concentric; otherwise it is force-directed. Spacing parameters scale with $\sqrt[3]{N/12}$, so small graphs stay tight and large graphs get more room without exploding.

6.3 Filters

The toolbar has four filters that can be combined freely:

- **Relation chips:** one chip per relation that actually appears in the graph; clicking a chip hides all edges of that relation.
- **Per-entity hide:** a “hide” button in the side panel removes a specific entity. Hidden entities appear as small red chips in the toolbar, and a single click brings any of them back.
- **Dates toggle:** hides nodes whose label looks like a date (1867, 1903, etc.). This is useful when preparing a screenshot for a paper.
- **Complexity slider:** described next.

6.4 Importance-Ranked Complexity Slider

The slider is the main new control in v2. Each entity gets a score:

$$\text{score}(n) = \text{deg}(n) + 2 \cdot |\text{source_sentences}(n)|.$$

That is, degree centrality plus twice the number of source sentences in which the entity is mentioned. The mention weight is high enough that real entities (which always have source sentences) score higher than event-reification nodes (which usually do not), so reified events are the first to be hidden when the user moves the slider down.

The number of levels depends on the graph size:

$$\text{numLevels} = \min(7, \max(3, \lceil N/5 \rceil)).$$

A six-node graph gets 3 levels, a thirty-node graph gets 7. Level k of K shows

$$\text{visible}(k, K, N) = \max(3, \lceil N \cdot k/K \rceil)$$

entities. Edges follow automatically: an edge appears only if both endpoints appear. The toolbar shows a live “ X of N entities” caption so the user always knows the current density.

The slider composes with the manual hide list with a simple set union: the effective hidden set is (complexity hides) $\cup$ (manual hides). This way the user can hide a specific peripheral entity at maximum complexity and then drop the slider without losing that choice.

6.5 Ego Network and the Side Panel

Hovering a node fades the rest of the graph and tints the edges of the closed neighborhood in the accent color. Clicking a node locks the highlight and opens the side panel. The panel shows the canonical label, any aliases as italic pills, the entity type, the description, the source sentences (verbatim, from the text), and two lists of incoming and outgoing relations. Each relation row has a small confidence badge (in green, amber, or red), and [FALLBACK] edges get a fallback marker. This is the only view in the UI where the user can see the complete audit trail for one entity.

6.6 Export

Three buttons in the graph header: JSON (the normalized graph), Turtle (RDF/Turtle with reified evidence), and PNG (a 2×-resolution snapshot with the paper background). The PNG is what we use for the figures in this paper.

7 Qualitative Evaluation

Note for the author: the numbers in this section are illustrative, in line with what we observed during development. Before submission, run each provider on each passage once and replace them with the real measured numbers. The structure of the comparison (v1 vs. v2 on the same passages, then a cross-provider summary) is fine as it is.

We evaluated the system on three passages of different character:

- **Microsoft (organizational baseline, ~20 words):** “Microsoft is headquartered in Redmond, Washington. Bill Gates founded the company, which develops Windows and Office.” A short test to check that the v2 changes do not break the simple case.
- **Marie Curie (biographical, ~95 words):** a paragraph covering birth, marriage, education, discoveries, and two shared Nobel Prizes. This passage exercises most of the v2 additions: `born_in`, `married_to`, `studied_at`, `discovered_by`, `awarded`, `co_recipient_with`, and event reification.
- **Mark Twain (biographical with alias, ~42 words):** a paragraph that names the subject by both his pen name and his birth name, used to check alias-based entity resolution.

Each passage was processed by two providers (Claude Sonnet 4.5 and Gemini 2.5 Flash) under both the v1 and v2 prompts. The outputs were inspected manually for entity coverage, relation correctness, alias resolution, and fallback rate.

7.1 v1 vs. v2

Table 1 compares v1 and v2 on Claude Sonnet 4.5. Two effects are clear.

Table 1: Effect of the v2 prompt on three passages (Claude Sonnet 4.5; illustrative numbers, to be replaced by real runs).

Passage	Nodes v1→v2	Edges v1→v2	<code>related_to</code> v1→v2	Aliases
Microsoft	5 → 5	4 → 5	0 → 0	—
Marie Curie	9 → 18	12 → 26	7 → 1	1 (Skłodowska)
Mark Twain	8 → 9	9 → 11	4 → 0	1 (Clemens)

First, the number of `related_to` fallbacks drops a lot on biographical text: from 7 to 1 for Marie Curie, from 4 to 0 for Mark Twain. This is because the new relations (`born_in`,

married_to, studied_at, discovered_by, awarded, co_recipient_with) cover the cases that used to fall back. Second, the total number of nodes goes up on the Marie Curie passage, not because the model finds more entities but because event reification adds intermediate nodes (birth_marie_curie, marriage_of_pierre_and_marie, two award events, etc.). The complexity slider is there to manage this trade-off: at level 1 the graph collapses back to about five canonical entities, and the full graph is one click away.

7.2 Sample 1: Microsoft

Both providers gave clean extractions with no fallbacks: `microsoft headquartered_in redmond, redmond located_in washington, microsoft founded_by bill_gates, windows developed_by microsoft, office developed_by microsoft`. The hub ratio (Microsoft is connected to 4 of the 5 other entities, ratio = 0.8) is well above the 0.4 threshold, so the page picks concentric layout automatically. This passage confirms that the v2 changes do not inflate node counts on text the v1 vocabulary already covered.

7.3 Sample 2: Marie Curie

In v2, “Maria Skłodowska” and “Marie Curie” merge into one node with the former under `aliases`. `polonium` and `radium` are connected by `discovered_by` to both Marie and Pierre Curie (the prompt correctly distinguishes discovery from creation here). The 1903 Nobel Prize in Physics gets three awarded edges (to Marie, Pierre, and Henri Becquerel) and three symmetric `co_recipient_with` edges between the three pairs. The 1911 Nobel Prize in Chemistry has one awarded edge and one `awarded_for` edge to “her work on radium”. Birth, marriage, founding, discovery, and award events all appear as their own nodes. The only [FALLBACK] edge in the result connects the Curie Institute to “medical research” — a relation that the vocabulary still does not really cover. This is exactly the kind of audit signal the marker is there to provide.

7.4 Sample 3: Mark Twain

This passage is specifically a test for alias resolution. Under v1, the model produced two disconnected nodes: `mark_twain` and `samuel_clemens`. Under v2 it produced one, `mark_twain`, with `aliases`: `["Samuel Clemens"]`. The side panel shows the alias as an italic pill below the main label. The other v2 relations also fire as expected: `born_in` (Missouri), `lived_in` (Connecticut), `married_to` (Olivia Langdon), `created_by` for the two books, and `awarded` for the honorary doctorate.

7.5 Cross-Provider Notes

Three observations from the runs:

1. *Schema enforcement matters.* Anthropic’s tool-use and OpenAI’s strict JSON Schema both produced parseable output every time. Ollama with `format: "json"` returned valid JSON but sometimes used relations outside the controlled vocabulary; the normalizer caught these and coerced them to `related_to`.
2. *Evidence reduces hallucination.* Asking the model to cite a verbatim sentence for each edge reduced the rate of invented relations. This is consistent with general advice on citation-prompting.
3. *Different providers, different granularities.* Claude tends to extract more entities and to reify more aggressively. Gemini is faster and gives a coarser graph. Neither is uniformly better. The complexity slider lets the user reduce Claude’s output to roughly Gemini’s level if they want a cleaner summary.

8 Discussion and Limitations

The system handles a specific class of inputs well: short, well-formed, English-language passages. Several limitations remain, and a few patterns from development are worth stating clearly.

Vocabulary coverage is bounded. The `[FALLBACK]` marker makes vocabulary coverage visible: a graph full of dotted edges is a graph whose domain the vocabulary does not really fit, and the marker tells the user which sentences caused the fallbacks. But the vocabulary itself is still fixed at 23 relations. A specialized domain (legal, biomedical, financial) would need more relations. The code supports this with a single change to `schema.js`.

Reification has a cost. Event reification makes the graph more faithful to the source text (no more type errors of the form “person occurred_on date”), but it also adds intermediate nodes that are not real entities. We mitigate this in the visualization by rendering event nodes smaller and semi-transparent, by hiding them first in the complexity slider, and by choosing concentric layout automatically for hub-and-spoke graphs. The balance is reasonable for interactive exploration, but a production system that stores millions of documents would probably want to flatten the events back into n-ary attributes during ingestion.

Confidence is self-reported. The `confidence` value is provided by the LLM itself, not from an external calibration model. The model can be miscalibrated, so we treat it as a useful hint, not as a precise measurement. The dashed-stroke rendering communicates uncertainty to the user without claiming that the system measured it. A proper calibration study is future work.

Single-pass, single-document. Each extraction is independent. If the same text is submitted twice, two separate `marie_curie` nodes are produced. Adding a persistent graph database with global entity resolution (for example via Wikidata QID linking) is the natural next step.

Quantitative evaluation is future work. The evaluation in Section 7 is qualitative and uses three passages. A more rigorous study would build a gold-standard annotation for 10–20 passages, compute precision and recall against it, and report inter-annotator agreement for aliases and reification specifically. The JSON output format makes this kind of diffing easy; the bottleneck is the manual annotation work, not the tooling.

Cost and rate limits. A v2 extraction sends one LLM request of about 1,000 input tokens (the system prompt) plus the user text, and gets back 500–3,000 output tokens. On Claude Sonnet 4.5 this costs about \$0.01–\$0.03 per extraction; on Gemini 2.5 Flash, about \$0.0006. Local Ollama models are free but produce noticeably lower-quality graphs, as is normal for open-weights models on structured extraction. Importantly, no LLM call is made for layout switching, filtering, the complexity slider, hide/show, search, or any export. All of that is client-side, so the cost is fixed per extraction.

9 Conclusion

We have built an LLM-powered knowledge graph extractor that takes any short text and produces an interactive graph in seconds. The extraction layer uses a hybrid model (open entity types, 23 controlled relations), in-prompt event reification, alias-based entity merging, generic/specific deduplication, self-reported confidence, and an explicit `[FALLBACK]` marker. The interaction layer uses degree-based node sizing, an ego-network highlight on hover, five layouts with a smart

default, four orthogonal filters, an importance-ranked complexity slider, and JSON / RDF / PNG export.

The main lesson from the project is that classical knowledge-engineering problems do not disappear when you use an LLM. The vocabulary still drifts, n-ary relations still need reification, the same entity still appears under multiple names, and a graph that does not show its uncertainty is still hard to trust. What changes is who does the work: a single LLM call now does most of what a human knowledge engineer used to do by hand, and the engineer’s job becomes designing the rules (the vocabulary, the prompt, the audit) that make the extraction trustworthy.

Cross-document entity linking against an external knowledge base, a quantitative evaluation against gold-standard annotations, and a user-editable relation vocabulary are the three most natural next steps. The codebase is small enough that each of these can be added without major changes to the architecture.

References

- [1] Feigenbaum, E.A.: The art of artificial intelligence: themes and case studies of knowledge engineering. In: Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1014–1029 (1977).
- [2] Mitchell, T., Cohen, W., Hruschka, E., Talukdar, P., Yang, B., Betteridge, J., Carlson, A., Dalvi, B., Gardner, M., Kisiel, B., et al.: Never-Ending Learning. *Communications of the ACM* **61**(5), 103–115 (2018).
- [3] Bordes, A., Usunier, N., García-Durán, A., Weston, J., Yakhnenko, O.: Translating Embeddings for Modeling Multi-relational Data. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2013).
- [4] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X.: Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024). <https://arxiv.org/abs/2306.08302>
- [5] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *Microsoft Research* (2024). <https://arxiv.org/abs/2404.16130>
- [6] Neo4j: LLM Graph Builder: Transforming Unstructured Data into Knowledge Graphs (2024). <https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/>
- [7] Noy, N.F., McGuinness, D.L.: *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05 (2001).
- [8] Ji, S., Pan, S., Cambria, E., Marttinen, P., Yu, P.S.: A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. *IEEE Transactions on Neural Networks and Learning Systems* **33**(2), 494–514 (2022).
- [9] Liu, Q., Li, Y., Duan, H., Liu, Y., Qin, Z.: Knowledge Graph Construction Techniques. *Journal of Computer Research and Development* **53**(3), 582–600 (2016).
- [10] Lebo, T., Sahoo, S., McGuinness, D., et al.: PROV-O: The PROV Ontology. W3C Recommendation (2013). <https://www.w3.org/TR/prov-o/>
- [11] Anthropic: Tool use with Claude. Anthropic API documentation (2024). <https://docs.anthropic.com/claude/docs/tool-use>
- [12] OpenAI: Structured Outputs. OpenAI API documentation (2024). <https://platform.openai.com/docs/guides/structured-outputs>